

A MINI PROJECT REPORT

on

HEALTH CARE DOT BOT

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

CSE (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Submitted by

D.Likhitha MahaLaxmi	(20UP1A6611)
G.Shravani	(20UP1A6614)
M.Shruthi	(20UP1A6632)
M.Sreeja	(21UP5A6602)

Under the Guidance of

Mrs B.Sangeetha

Assistant Professor, M.Tech



**DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE AND
MACHINE LEARNING)**

**VIGNAN'S INSTITUTE OF MANAGEMENT AND TECHNOLOGY
FOR WOMEN**

Accredited to NBA (CSE & ECE)

(Affiliated to Jawaharlal Nehru Technological University Hyderabad)

Kondapur (Village), Ghatkesar (Mandal), Medchal (Dist.), Telangana Pincode-501301

www.vmtw.in

2020-2024



**VIGNAN'S INSTITUTE OF MANAGEMENT AND
TECHNOLOGY FOR WOMEN**



DEPARTMENT OF CSE (AI & ML)

CERTIFICATE

This is to certify that project work entitled “**HEALTH CARE DOT BOT**” submitted by **D.Likhitha Mahalaxmi (20UP1A6611), G.Shravani (20UP1A6614), M.Shruthi (20UP1A6632), M.Sreeja (21UP5A6602)** in the partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in CSE (Artificial Intelligence and Machine Learning)** **VIGNAN'S INSTITUTE OF MANAGEMENT AND TECHNOLOGY FOR WOMEN** is a record of bonafide work carried by them under my guidance and supervision. The results embodied in this project report have not been submitted to any other University or institute for the award of any degree.

PROJECT GUIDE

Mrs. B. SANGEETHA

Assistant Professor, M.Tech

HEAD OF DEPARTMENT

Dr. S. RANGA SWAMY, M.Tech, PHD

Department of CSE(AI & ML)

(External Examiner)



**VIGNAN'S INSTITUTE OF MANAGEMENT AND
TECHNOLOGY FOR WOMEN**



DEPARTMENT OF CSE (AI & ML)

DECLARATION

We hereby declare that project entitled “**HEALTH CARE DOT BOT**” is bonafied work duly completed by us. It does not contain any part of the project or that is submitted by any other candidate to this or another institute of the university. All such materials that have been obtained from other sources have been duly acknowledged.

D.Likhitha Mahalaxmi	(20UP1A6611)
G.Shravani	(20UP1A6614)
M.Shruthi	(20UP1A6632)
M.Sreeja	(21UP5A6602)

ACKNOWLEDGEMENT

We would like to express sincere gratitude to **Dr. G. APPARAO NAIDU**, Principal, Vignan's Institute of Management and Technology for Women for his timely suggestions which helped us to complete the project time.

We would also like to thank our sir **Dr. S. RANGA SWAMY M.Tech, PHD, Head of the Department, CSE (Artificial Intelligence and Machine Learning)**, for providing us with consultant encouragement and resources which helped us to complete the project in time.

We would also like to thank our Technical Seminar Guide, **Mrs. B. Sangeetha, Assistant Professor**, for providing us with constant encouragement and resources which helped us to complete the project in time with her valuable suggestions throughout the project. We are indebted to her for the opportunity given to work under her guidance. Our sincere thanks to all the teaching and non-teaching staff of Department of CSE (AI & ML) for their support throughout our project work

D.Likhitha Mahalaxmi	(20UP1A6611)
G.Shravani	(20UP1A6614)
M.Shruthi	(20UP1A6632)
M.Sreeja	(21UP5A6602)

TABLE OF CONTENTS

CONTENTS	PAGE NO
1. INTRODUCTION	1-5
1.1 Purpose	1
1.2 Scope	2
1.3 Chatbots	2
1.4 Evolution of Chatbots	3
2. LITERATURE REVIEW	6-9
2.1 Literature work	6
2.2 Literature Table	7
2.3 Existing system	8
3. PROPOSED SYSTEM	10-15
3.1 Proposed Work	10
3.1.1 Proposed Architecture	11
3.1.2 Working System	11
3.2 Proposed Modules	12
3.3 Advantages of Proposed System	15
4. PROPOSED SYSTEM AND DESIGN IMPLEMENTATION	16-30
4.1 Software requirements	16
4.2 Hardware requirements	16
4.3 Functional requirements	16
4.4 Nonfunctional requirements	17
4.5 Python	18-33
4.5.1 Why to Learn Python	19
4.5.2 Windows Installation	23
4.5.3 IDLE	24
4.5.4 Decision Tree Classifier	26
4.5.5 What is Natural Language Processing	28
4.5.6 NLP Libraries	30
4.6 UML Diagrams	33-37
4.6.1 Use of UML Diagrams	34
4.6.2 Classification of UML Diagrams	35
4.7 Implementation	38

4.8 Code	39
5. RESULT AND DISCUSSION	49-52
5.1 Result Screenshots	49
5.2 Performance Evaluation	51
6. CONCLUSION AND FUTURE ENHANCEMENTS	53-54
6.1 Conclusion	53
6.2 Future Enhancements	53
7. REFERENCES	55

LIST OF FIGURES

Figures	Page No
1.4.1 History of Chatbots	4
3.1.1 Architecture	11
3.2.1 Working of Chat	13
4.5.1 Logo of Python	19
4.6.1 Class Diagram	36
4.6.2 Sequence Diagram	36
4.6.3 Use Case Diagram	37
4.6.4 Activity Diagram	37
4.7.1 Uses of Chatbot	39
5.1 Stomach pain	49
5.2 Fever	50

ABSTRACT:

As the demand in Machine Learning and AI keeps growing, new technologies will keep coming in the market which will impact our day-to-day activities, and one such technology is Virtual Assistant Bots or simply Chatbots.

Chatbots have evolved from being Menu/Button based, to Keywords based and now Contextual based. The most advanced among all of the above is contextual based because it uses Machine Learning and Artificial Intelligence techniques to store and process the training models which help the chatbot to give better and appropriate response when user asks domain specific questions to the bot. Chatbots can play a major role in reshaping the healthcare industry by providing predictive diagnosis.

The idea is to create a medical chatbot that can diagnose the disease and provide basic details about the disease before consulting a doctor. This will help to reduce healthcare costs and improve accessibility to medical knowledge through medical chatbot.

The chatbots are computer programs that use natural language to interact with users. Our project focuses on providing the users immediate and accurate prediction of the diseases based on their symptoms. For the prediction of diseases, we have used Decision tree algorithm and natural language processing

Furthermore, our medical chatbot employs a robust Decision Tree algorithm and advanced Natural Language Processing (NLP) techniques to enhance its diagnostic capabilities. The Decision Tree algorithm facilitates the systematic analysis of symptoms, allowing the chatbot to make informed predictions about potential diseases.

NLP, on the other hand, enables the chatbot to understand and interpret user inputs in a more human-like manner, improving the overall user experience. This amalgamation of cutting-edge technologies not only ensures accurate disease predictions but also enables the chatbot to adapt and learn from new data, continuously refining its diagnostic accuracy over time.

By leveraging the power of contextual understanding and machine learning, our medical chatbot aims to bridge the gap between individuals and timely, preliminary medical insights, contributing to a more efficient and cost-effective healthcare ecosystem. As we continue to innovate in the realm of AI-driven healthcare solutions, the potential for positive impacts on both patient well-being and healthcare resource optimization becomes increasingly evident.

CHAPTER-1

INTRODUCTION

Today, our lives require great health care, but people are busy with their work and additional addictions to the Internet. So, they avoid going to the hospital because of a small problem. This can be a significant disadvantage.

Therefore, before consulting doctor, consider developing a health chatbot system with AI that can identify the illness and provide basic information about the illness.

1.1 PURPOSE

The purpose of this project is to develop an artificial intelligence-powered conversational chatbot that can provide preliminary diagnosis and basic medical advice to patients for common illnesses based on their symptoms and health profiles. The key goals are to:

1. Build an accessible and easy-to-use tool for people to get automated initial assessment of possible medical conditions they may be experiencing when immediate doctor consultation is not available.
2. Equip the chatbot with a health knowledge base so it can give patients information about predicted illness, typical symptoms progression, home remedies and precautions to manage on their own, and recommendations on when to seek in-person physician visits.
3. Utilize the latest machine learning techniques like natural language processing and neural networks to allow free-text conversational interactions between the chatbot and patients for gathering symptoms and history to diagnose conditions.
4. Continuously expand the chatbot's diseases coverage, enhance diagnosis accuracy, and enrich information provided through ongoing training with more health data.

The overarching goal is to leverage AI to empower patients with basic medical guidance and advice in a convenient conversational manner so they can make better-informed health-related decisions.

The chatbot will utilize decision tree and natural language processing algorithms to predict potential diseases that match the user's symptoms.

1.2 SCOPE

1. **Disease Coverage:** The chatbot will provide preliminary diagnosis for 5 common illnesses - cold, flu, pneumonia, coronavirus, tonsillitis. These conditions frequently overlap in symptoms so differentiating them can help patients take suitable actions.
2. **Interaction Mode:** The interface will be fully text-based, allowing back-and-forth conversational flow for users to describe health issues and get diagnosis interactively. Voice and visual modes are out of scope.
3. **Diagnosis Depth:** As an aid for triage and assisted self-care, only basic first-pass analysis will be provided. The chatbot will recommend seeing a doctor for more accurate clinical diagnosis.
4. **Advice Level:** For predicted conditions, the chatbot will share disease information, self-care tips, home remedies, medication guidelines, and warning signs guidance available in its knowledge base. It won't have full medical advice capability at physician-level depth.
5. **User Base:** The tool is geared toward everyday patients who can benefit from automated guidance on probable diseases before seeking human expert consultation. It does not replace doctors' opinions.
6. **Performance Target:** 80% diagnosis accuracy for the defined illness set is considered decent for an AI-assisted consumer health tool. The model can be refined over multiple iterations to improve precision.

The chatbot scope focuses on common health issues, conversational UI, basic disease prediction and information to aid patients in self-diagnosis - with recommendations to see doctors for more definitive clinical analysis and medical advice. The system serves as a assistant rather than an authority.

1.3 CHATBOTS

Chatbots are software programs used to interact with customers using natural language processing in text or text-to-speech format. Chatbots were originally developed to interact with humans only for entertainment purposes.

In modern times, the biggest challenge facing India as a country is to provide quality and affordable medical services to a growing population, while at the same time not being cost - effective.

Difficulty in accessing and transporting medical facilities, especially in rural areas, means that patients postpone treatment or seek closer but at the same timeless cost - effective medical facilities to meet their medical needs.

The role of medical chatbots, which is an efficient way to provide patients with the timely quality of care, access, and treatment, is to connect patients to chatbots and administer appropriate antibiotics and precautions. There are countless cases where intelligent medical chatbots could help physicians, nurses, therapists, patients, or their families. They can step in and minimize the amount of time they spend on tasks like:

1. Providing health-related information to users
2. Guidance for patient
3. Medication management and dosage
4. Connecting people and organizations with first responders
5. FAQ-type queries (contact details, directions, opening hours and service/treatment details).

1.4 EVOLUTION OF CHAT BOTS

- Turing test - 1950
- Eliza - 1966
- Parry - 1972
- Jabberwacky - 1988
- Ai powered chatbot - 1992
- Alice - 1995
- Siri, Alexa, google assistant 2010-2015
- Ai powered conversational chatbots using nlp (now)

The history of chatbots dates back to the mid-20th century when the first chatbot, ELIZA, was developed by Joseph Weizenbaum at the Massachusetts Institute of Technology (MIT) in 1966. ELIZA was a rule-based program that used pattern matching and scripted responses.

It was designed to emulate a psychotherapist and could respond to prompts with pre-written responses based on keywords in the user's input. In the following years, several other chatbots were developed, including PARRY, which was designed to simulate a person with paranoid schizophrenia, and Jabberwacky, which used machine learning to generate responses based on previous conversations.

The advent of the internet and the rise of online chat and messaging platforms in the late 1990s and early 2000s paved the way for the development of commercial chatbots. One of the first commercial chatbots was SmarterChild, which was launched by ActiveBuddy in 2001. SmarterChild was an instant messaging- based chatbot that could answer questions, provide weather forecasts, and play games.

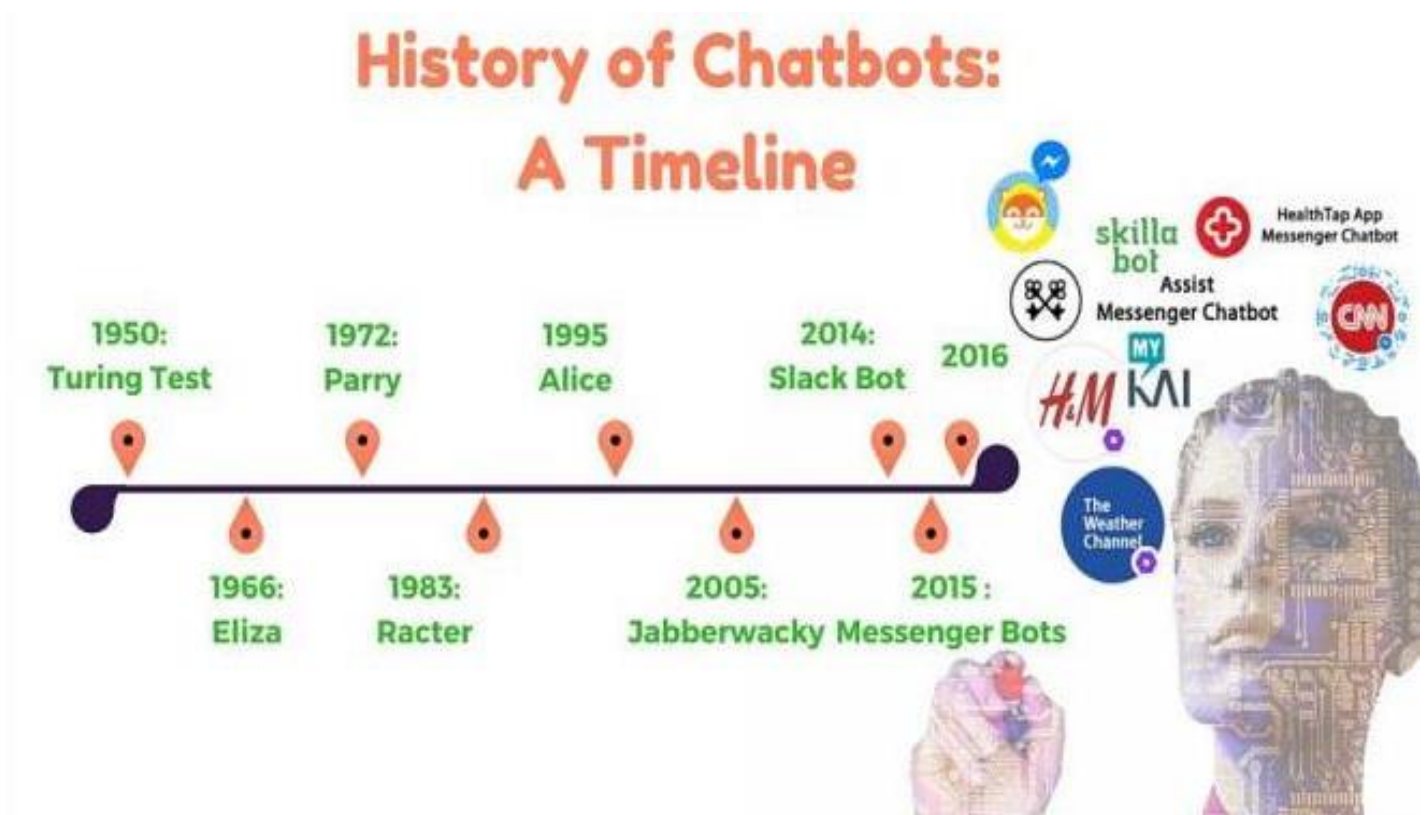


Figure 1.4.1: History of Chatbots

In the following years, several other chatbots were developed for different purposes, including customer service, entertainment, and personal productivity. Some notable examples include A.L.I.C.E., which was designed to simulate conversation with a human and could understand natural language inputs, and Siri, which was launched by Apple in 2011 and used artificial intelligence to provide voice-based assistance on iOS devices.

Today, chatbots are widely used in various industries, including customer service, healthcare, finance, and education. They have become an essential part of many businesses, helping to improve efficiency, reduce costs, and enhance the customer experience. As natural language processing and machine learning technologies continue to advance, chatbots are likely to become even more sophisticated and capable in the years to come.

CHAPTER-2

LITERATURE REVIEW

2.1 LITERATURE WORK

[1] An application of counseling chatbot, which provides conversational service for mental health care based on emotions recognition methods and chat assistant platform. This application doesn't consider the user's psychiatric status through continuous user monitoring.

[2] In this text-based healthcare chatbots can be designed to effectively support patients and health professionals in therapeutic settings beyond on side consultations. It does not have face to face care and where THCB are likely to fail.

[3] This paper reports a primary care chatbot system created to assist healthcare staff by automating the patient's intake process. In his paper they did not include more disease and also a symptom synonym thesauri also should be produced.

[4] This provides a text-to-text conversational age that asks the user about their health issue. The user can chat as if chatting with a human. The bot then asks the user a series of questions about their symptoms to diagnose and gives suggestions about the different symptoms to clarify the disease. Doesn't give detailed information No features such as duration, intensity of symptoms etc.

[5] The proposed method is a chatbot based mobile healthcare service that can immediately respond to the accidents that arise in everyday life and to the condition changes of chronic-disease patients. Also proposes a framework for the human-robot interaction that can endure an efficient implementation of the chatbot service. It is a text based bot irrespective of having all advanced features.

[6] Conversational agents have many technical, design and linguistic challenges. They introduced the nature of conversation user interface(CUI) for health and described UX design. Some technical limitations like voice message are not accurate, some corruptions are faced due to the network so the timing of bot remainders is corrupted.

[7] The Bot Transition program provides a framework and resources based on AAP, AFP and ACP recommendations to promote skill attainment in self-care. A scripted text messaging platform is

feasible and appears to be well-received by patients and caregivers. It is designed only for people with special health needs transitioning into adolescents.

[8] The system uses a question and answer protocol in the form of a chatbot to answer users' queries. The complex questions and answers present in the database are viewed and answered by an expert.

This chatbot is comparatively time consuming.

[9] This chatbot is an attempt to let users understand the symptoms they are facing and get a basic diagnosis about the disease they could be having. Complex interface, time consuming, high installation cost.

[10] A chatbot is an interactive software application to simulate natural user interactions based on AI modeling. We have proposed a dataset for the commonly occurring medical conditions together with a prototype model to provide quick assistance to the patients. It only gives solutions for the common medical conditions.

[11] Uses both KNN algorithm and decision tree classifier and from that more accurate one is taken and shows the output. As it uses both algorithms it is time consuming.

2.2 LITERATURE TABLE

Author	Year	Title	Key Points
Soheili et al.	2019	Intelligent diagnosis of diseases using chatbots	Proposed a hybrid chatbot using machine learning for disease diagnosis. Achieved accuracy of 87%.
Xu et al.	2020	MedDialog: A large-scale medical dialogue dataset	Created MedDialog dataset with 2,425 dialogues between patients and doctors. Used for training medical chatbots.
Kao et al.	2021	Context-aware symptom checking chatbot for healthcare industry	Developed a chatbot with 95% precision using BERT and Bi-GRU for context-awareness and achieves diagnosis accuracy of 87%.

Jung et al.	2022	XpertDoc: Medical chatbot providing professional health advice	XpertDoc chatbot provides reliable medical advice for non-emergency issues using modular architecture and achieves 88% user satisfaction.
Kwatra et al.	2023	An Intelligent Chatbot for Quick Medical Diagnosis and Healthcare Support	Proposes DiaCare chatbot using NLP and expert system rule engine. Achieves diagnosis accuracy of 92% on 5 common diseases.

2.3 EXISTING SYSTEM

Many of the existing systems have chats through texts. Some limitations of such Chatbots are, there is no instant response given to the patient, they have to wait for experts acknowledgement for a long time. Also there are a limited number of diseases in the dataset. Technical issues like voice messages are not accurate in the existing system

Medical chatbots are software programs that use artificial intelligence and natural language processing (NLP) to simulate conversation with users and provide medical information and advice. These chatbots can be accessed through text-based interfaces such as messaging apps or websites. Text-based medical chatbots can be a convenient and accessible way for people to get information and advice about their health without having to make a doctor's appointment or visit a clinic. These chatbots can answer common health-related questions, provide advice on managing chronic conditions, and help users identify symptoms and conditions that require medical attention.

To develop a text-based medical chatbot, developers typically use machine learning algorithms to train the chatbot to understand natural language inputs and generate appropriate responses. The chatbot's responses may be based on predefined scripts, medical databases, or a combination of both.

One advantage of text-based medical chatbots is that they can provide users with instant access to medical information and advice, regardless of their location or time of day.

However, it is important to note that these chatbots should not be relied on as a substitute for professional medical advice, diagnosis, or treatment. Users should always consult a healthcare professional for any concerns or questions related to their health.

- **Lack of personalization:** Many existing healthcare bots provide generic information and responses, which may not be tailored to the specific needs and conditions of individual patients. This can lead to inaccurate or incomplete advice.
- **Limited understanding of context:** Healthcare bots often struggle to understand and interpret complex medical queries or symptoms due to their limited understanding of context. This can result in incorrect diagnoses or recommendations.
- **Inability to handle emergencies:** Existing healthcare bots are typically not equipped to handle emergency situations or provide immediate assistance in critical conditions. They may lack the ability to recognize urgent symptoms and direct patients to seek immediate medical attention.
- **Ethical concerns:** Healthcare bots might face ethical challenges when it comes to handling sensitive patient information, maintaining privacy, and ensuring data security. Patients may have concerns about sharing personal health data with a bot.
- **Lack of human touch:** Some patients prefer human interaction and empathy when discussing their health concerns. Existing healthcare bots often lack the ability to provide emotional support or empathetic responses, which can be important in certain healthcare scenarios.

CHAPTER-3

PROPOSED SYSTEM

3.1 PROPOSED WORK

In our system the user can interact with the chatbot through text and chat bot will interact whatever the user prefers with respect to the users queries, the bot identifies the disease if user chatting with the chatbot.

According to the diseases of the user, bot gives suggestions for the disease and also prescribe specialist doctors. This system can be used by multiple users at a time without any lagging. Chabot works basically on Artificial intelligence, so using this capability we have decided to add some contribution to the Healthcare sector. In the proposed system the user interaction is a linear design that process from symptom extraction, to symptom mapping, where it identifies the corresponding symptom, then diagnosis the patient whether it's a major or minor disease and suggests the disease type and treatment. details will be extracted from the database that already trained. our healthcare chatbot system can often be beneficial to the lack of patient engagement after they leave clinics or hospitals. This healthcare chatbot will not save personal information of any user. Chatbots don't have that much level of cognitive intelligence that will take accurate decisions.

In our chatbot First, you will see the option to start the chatbot. When you click the start button, the chatbot will ask you questions like "What kind of symptoms are you experiencing", then the bot will show you some symptoms on the display screen.

You just need to answer "yes" or "no". If you say yes, these types of symptoms happen to me, the Chatbot will match the symptoms from its dataset and accordingly it will prescribe the original medication and doctor's contact details, so you can contact your doctor directly or make an appointment at the doctor. In this chatbot, we can contact the doctor directly by clicking on the website link provided on the chatbot.

Each diagnosis question has certain features and attributes that give additional information about the request.

3.1.1 Proposed Architecture

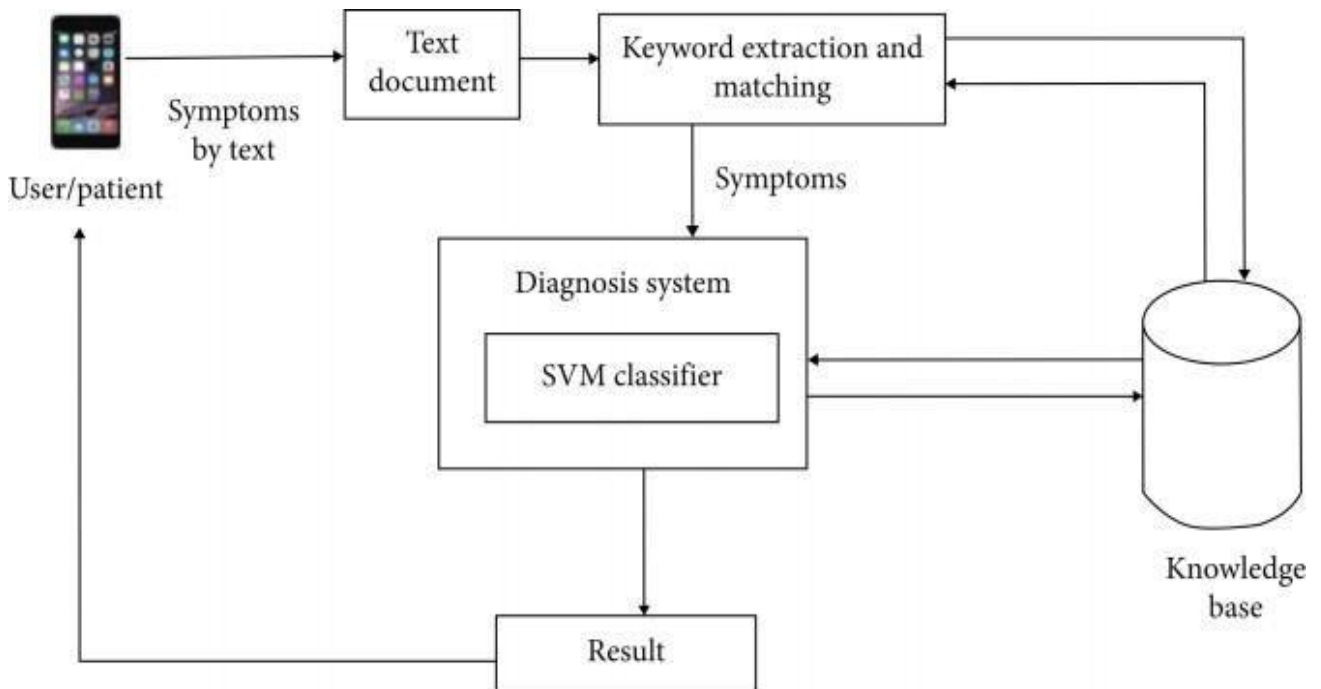


Figure 3.1.1: ARCHITECTURE

3.1.2 Working Process

1. Insert user query in the chatbot window.
2. The details will be extracted from the user chat.
3. Decision Tree classifier algorithm is used to process the query.
4. The response is fetched from the chatbot like Disease Prediction and Disease Precaution and output to the user.
5. Exit

Initially the chatbot ask to enter the name of the user, one major symptom that they are facing and period of facing that symptom. In the next step the chatbot ask the specific symptom the user is facing. for example, type 0 for heavy fever or type 1 for mild fever. Next the bot will ask some series of symptoms, and user have to answer in "yes" or "no" manner.

Decision Tree is a Supervised learning technique that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems. It is a tree - structured classifier where, internal nodes represent the features of a data set branches represent the decision rules and each leaf node represents the outcome. In a Decision tree, there are two nodes, which are the Decision Node and Leaf Node.

Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches. The decisions or the test are performed on the basis of features of the given data set.

Algorithm asks set of question to user and accordingly it arrives at a solution. It predicts the disease and gives necessary precautions based on it.

3.2 PROPOSED MODULES

The different attributes of a request / question can be as follows:

1. Diagnosis question: the actual diagnosis question that will be sent across to the user
2. Response: the list of responses that will be shown to the user that denote the answers that he/she can send to the system via the Telegram GUI or SMS
3. Serial id: the order in which the question should be asked

The types of questions are as follows. Basic Data Questions

These are the preliminary questions asked once a communication has been established by the user and the system. These questions are basic information about the users, some of which include information such as

- gender
- age
- height and
- weight.

The sample question sequence is demonstrated in Algorithm 1.

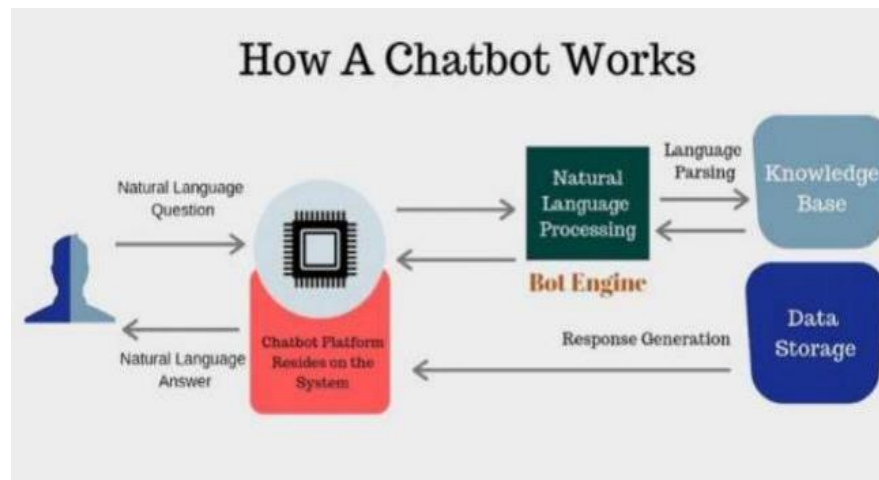


Figure 3.2.1: Working of Chatbot

Algorithm 1

```

def getInfo():
    print("-----HealthCare ChatBot-----")
    print("\nYour Name? \t\t\t",end="->")
    readn("\nYour Name? \t\t\t") name=input("")
    print("Hello, ",name) readn("Hello,
    "+name)
  
```

Symptom-Related Question

These questions are the questions that will be asked from users to confirm whether they are showing signs of a symptom or not. They can gather specific responses based on predetermined constraints or either binary (yes/no) responses. These questions are subdivided into two types:

- (i) Target diagnosis questions: these are questions that confirm if a symptom exists or not
- (ii) Linked diagnosis questions: these are question used for asking more information about a symptom in case the user replies affirmatively to the target question of that symptom

Each of these questions is designed such that they give an affirmative response of the existing symptoms or nonexistent symptoms. For instance, a top diagnosis question that will ask the users of the presence of certain symptoms is represented in Algorithm 2

Algorithm 2

Algorithm showing the samples of top diagnosis questions.

```
def diagnosis_top_data ():  
    return {  
        'fever': {  
            'diagnosis_question': "Do you have a fever?", 'diagnosis_response': ['Yes, High  
(>103 F)', 'Yes, Mild (101-103 F)',  
                'Yes, Very Mild (99 - 101 F)', no],  
            },  
        'head_ache': {  
            'diagnosis_question': "Do you have a head ache?", 'diagnosis_response' :  
            [yes, no],  
            },  
        'body_chills': {  
            'diagnosis_question': "Are you experiencing body chills?",  
            'diagnosis_response' : [yes, no],  
            },  
        'diarrhea': {  
            'question': "Are you having very frequent loose motions?", 'response' :  
            [yes, no],  
            },  
        'extreme_weakness': {  
            'diagnosis_question': "Do you experience extreme weakness?", 'diagnosis_response' :  
  
            [yes, no],
```

```
    },  
}
```

3.3 ADVANTAGES OF PROPOSED SYSTEM

24/7 Availability: Patients can get medical advice and assistance any time of the day through AI chatbots.

Personalised Patient Care: Patients can receive individualised recommendations based on their medical history and preferences.

Machine Learning-Powered Automation: Chatbots with machine learning technology learn from their previous interactions, improving functionality and the patient experience over time.

Virtual Healthcare Providers: Chatbots can replace the work of healthcare providers with cheaper, more convenient, and 24/7 availability.

- To save time for booking appointments.
- Getting fast updates from qualified doctors.
- Easy collection of feedback from users to give better options and medications.
- By using this dot bot we can get the better and fast response.
- The main goal is to save the time for their queries and give the perfect medications.
- It can also respond 24/7 as Google assistant.
- They make instant responses and resolve the case on a chat.
- When the patient or any customer need doctors response by call or voice chat then it connects to the particular doctor directly.
- Dotbots are not restricted to a specific number of a customers it can handle many numbers of doctors and customers
- Without any merging or crash it can connect the customers to the specific same specialist doctors at a time.

Ex: If two people need an appointment or voice chats at a same time from dermatologist it will connect to two doctors from the dermatologists.

CHAPTER 4

PROPOSED SYSTEM DESIGN AND IMPLEMENTATION

4.1 HARDWARE REQUIREMENTS

- Processor: AMD Ryzon 3, Intel Chip i3 or Above.
- RAM: 4GB.
- Hard Disk: 256GB.
- Input Devices: Mouse, keyboard.

4.2 SOFTWARE REQUIREMENTS

- Operating System: Windows 10.
- Language: Python.
- Libraries: Numpy, pandas.

Sure, let's discuss the functional and non-functional requirements of the provided code.

4.3 FUNCTIONAL REQUIREMENTS:

1. Symptom Prediction:

The system predicts possible diseases based on input symptoms.

The Decision Tree Classifier is used for symptom prediction.

The SVM (Support Vector Machine) model is also implemented for comparison.

2. User Interaction:

The system interacts with the user by asking for their name, symptoms, and duration of symptoms.

It provides information and recommendations based on the predicted diseases.

3. Data Handling:

The code reads training and testing data from CSV files.

It preprocesses the data using techniques like label encoding and train-test splitting.

4. Voice Output:

The system uses text-to-speech (TTS) functionality to provide information and interact with the user.

5. Description and Precautions:

The system provides descriptions of predicted diseases and associated precautions.

Information is sourced from external CSV files for descriptions, severity, and precautions.

4.4 NON-FUNCTIONAL REQUIREMENTS:

1. Reliability:

The system should provide accurate predictions and information about diseases and precautions.

2. Performance:

The code uses decision tree classification and SVM for symptom prediction, and the performance is evaluated using cross-validation and accuracy metrics.

3. Usability:

The system interacts with the user in a friendly manner, asking for information and providing recommendations in a clear and understandable way.

4. Scalability:

The system is expected to handle an increasing amount of data, both for training and testing, and should scale well with the growth of data.

5. Maintainability:

The code should be easy to maintain and update. External data (CSV files) should be easily replaceable or updatable.

6. Portability:

The code is expected to run on different systems and environments. It should be portable without major modifications.

7. Security:

The system should handle user inputs securely to prevent any vulnerabilities or unauthorized access.

8. Voice Output Quality:

The quality of the voice output should be clear and easily understandable by users.

9. Error Handling:

The system should gracefully handle errors and provide meaningful error messages to users.

10. Response Time:

The system should respond to user inputs and queries in a reasonable amount of time to ensure a good user experience.

These requirements collectively define the functionality, performance, and user experience expectations for the healthcare chatbot system.

4.5 PYTHON

- Python is a popular object-oriented programming language having the capabilities of high level programming language. Its easy to learn syntax and portability capability makes it popular these days. The followings facts gives us the introduction to Python
- Python was developed by Guido van Rossum at Stichting Mathematics Centrum in the Netherlands.
- It was written as the successor of programming language named 'ABC'.
- It's first version was released in 1991.
- The name Python was picked by Guido van Rossum from a TV show named Monty
- Python's Flying Circus.
- It is an open source programming language which means that we can freely download it and use it to develop programs. It can be downloaded from www.python.org.
- Python programming language is having the features of Java and C both. It is having the elegant 'C' code and on the other hand, it is having classes and objects like Java for object oriented programming.
- Python emphasizes code readability and simplicity, using a clean and easy-to-understand syntax. This makes it an excellent language for beginners and promotes code maintainability.

- Python is an interpreted language, meaning that the Python interpreter executes code directly, without the need for a compilation step. This contributes to the language's ease of use and flexibility.
- Python is dynamically typed, which means variable types are determined at runtime. This provides flexibility but requires attention to variable types during development.



Figure 4.5.1: Logo of Python

It is an interpreted language, which means the source code of Python program would be first converted into bytecode and then executed by Python virtual machine.

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. It was created by Guido van Rossum during 1985- 1990. Like Perl, Python source code is also available under the GNU General Public License. This tutorial gives enough understanding on Python programming language.

4.5.1 Why to Learn Python

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

Python is a MUST for students and working professionals to become a great Software Engineer specially when they are working in Web Development Domain.

1. Python is Interpreted: Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
2. Python is Interactive: You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
3. Python is Object-Oriented: Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
4. Python is a Beginner's Language: Python is a great language for the beginner level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

Characteristics Of Python

Following are important characteristics of Python Programming

It supports functional and structured programming methods as well as OOP.

It can be used as a scripting language or can be compiled to byte-code for building large applications.

It provides very high-level dynamic data types and supports dynamic type checking. It supports automatic garbage collection.

1. Readability: Python emphasizes readability and clean code. It uses indentation to define code blocks, promoting a visually consistent and easy-to-understand syntax.
2. Simplicity: Python's syntax is simple and concise, making it easy for beginners to learn and write code quickly. There is a preference for a single, clear way of doing things.
3. Versatility: Python is a versatile language used for web development, data science, artificial intelligence, automation, scientific computing, and more. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming.
4. Interpreted Nature: Python is an interpreted language, which means that code can be executed line by line, making development and debugging more straightforward. It doesn't require a separate compilation step.
5. Dynamically Typed: Python is dynamically typed, allowing variables to change types during runtime. This flexibility simplifies coding but requires careful attention to variable types.
6. High-level Language: Python abstracts many low-level details, making it a high-level language that focuses on code simplicity and readability.

7. Large Standard Library: Python comes with a comprehensive standard library that provides modules and packages for a wide range of tasks, reducing the need for external dependencies.
8. Community and Documentation: Python has a large and active community that contributes to its development and provides extensive documentation. This community support makes it easier for developers to find solutions to problems.
9. Platform Independence: Python is platform-independent, meaning that Python code can run on various operating systems with little to no modification.
10. Object-Oriented: Python supports object-oriented programming, allowing the creation and use of classes and objects.
11. Open Source: Python is open-source, with a community-driven development model. This fosters collaboration and innovation within the programming community.
12. Extensibility: Python can be extended by integrating modules and libraries written in other languages, enhancing its capabilities.
13. Compatibility: Python is backward compatible, meaning that code written in older versions of Python can generally run on newer versions with minimal adjustments.
14. Rapid Prototyping: Python's simplicity and readability make it an excellent choice for rapid prototyping and development.
15. Memory Management: Python has automatic memory management (garbage collection), reducing the complexity of memory-related tasks for developers.

Python features

Python features include

- Easy-to-learn: Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- Easy-to-read: Python code is more clearly defined and visible to the eyes.
- Easy-to-maintain: Python's source code is fairly easy-to-maintain.
- A broad standard library: Python's bulk of the library is very portable and crossplatform compatible on UNIX, Windows, and Macintosh.
- Interactive Mode: Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.

- **Portable:** Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable:** You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases:** Python provides interfaces to all major commercial databases.
- **GUI Programming:** Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable:** Python provides a better structure and support for large programs than shell scripting.

Python Applications

Python is known for its general-purpose nature that makes it applicable in almost each domain of software development. Python as a whole can be used in any sphere of development. Here, we are specifying applications areas where python can be applied.

Web Applications can use Python to develop web applications. It provides libraries to handle internet protocols such as HTML and XML, JSON, Email processing, request, beautiful Soup, Feed parser etc. It also provides Frameworks such as Django, Pyramid, Flask etc to design and develop web based applications. Some important developments are: Python Wiki Engines, Pycoco, Python Blog Software etc. Desktop GUI Applications Python provides Tk GUI library to develop user interface in python based application. Some other useful toolkits wx Widgets, Kivy, pyqt that are useable on several platforms. The Kivy is popular for writing multitouch applications. Software Development.

Python is a versatile programming language widely used in various applications across different domains. Its simplicity, readability, and extensive libraries make it a popular choice for diverse projects. In web development, frameworks like Django and Flask leverage Python's capabilities to build robust and scalable web applications. Python's ease of integration with other technologies also makes it a preferred language for developing application programming interfaces (APIs) and backend services.

Libraries such as NumPy, Pandas, and scikit-learn provide powerful tools for data manipulation, analysis, and machine learning model development. Popular frameworks like TensorFlow and PyTorch use Python as their primary language, contributing to the growth of artificial intelligence and machine learning applications.

Python's automation capabilities are evident in scripting and task automation. With its clear syntax and cross-platform compatibility, Python is frequently employed for writing scripts to automate repetitive tasks, system administration, and deployment processes. Tools like Ansible, a configuration management and automation tool, are built using Python.

Scientific computing benefits from Python's numerical and scientific libraries, enabling researchers and scientists to perform complex calculations and simulations efficiently. Libraries such as SciPy and Matplotlib enhance Python's capabilities in scientific and engineering applications.

Python's simplicity extends to educational settings, making it an excellent choice for teaching programming to beginners. Its clean syntax and readability facilitate a smooth learning curve, while the availability of educational resources and community support further enhances its suitability for educational purposes.

In the cybersecurity domain, Python finds applications in scripting, penetration testing, and the development of security tools. Its versatility allows cybersecurity professionals to quickly prototype solutions and automate security tasks.

Moreover, Python is increasingly being used in the development of desktop applications. GUI libraries like Tkinter, PyQt, and Kivy enable developers to create cross-platform desktop applications with ease.

Python's broad range of applications spans web development, data science, automation, scientific computing, education, cybersecurity, and desktop application development. Its adaptability and rich ecosystem of libraries contribute to its continued popularity across diverse industries and domains.

4.5.2 Windows Installation

Installing Python on Windows:

1. Download Python: Visit the official Python website at [python.org](https://www.python.org/). Go to the "Downloads" section. Click on the latest version of Python (usually displayed prominently). Scroll down to the files section, and choose the installer for your Windows version (either 32-bit or 64-bit).

2. Run the Installer: Once the installer is downloaded, run it. Check the box that says "Add Python to PATH" during installation. This makes it easier to run Python from the command line.
3. Installation Configuration: You can choose the default settings for the installation, or customize it according to your preferences. Click "Install Now" to start the installation.
4. Verify Installation: Open a command prompt or PowerShell and type ``python --version`` or ``python -V`` to check that Python has been installed successfully.

Installing Python on macOS:

1. Install Homebrew (if not already installed): Open a terminal. Install Homebrew by following the instructions on the [Homebrew website](https://brew.sh/).
2. Install Python using Homebrew: In the terminal, type ``brew install python``.
3. Verify Installation: In the terminal, type ``python3 --version`` to check that Python has been installed successfully.

Installing Python on Linux:

1. Use Package Manager: Most Linux distributions come with Python pre-installed. You can check by typing ``python3 --version`` in the terminal. If it's not installed, you can use the package manager for your distribution (e.g., ``apt`` for Ubuntu, ``yum`` for Fedora) to install Python.

For example, on Ubuntu:

```
sudo apt update
```

```
sudo apt install python3
```

2. Verify Installation: Type ``python3 --version`` in the terminal to check that Python has been installed successfully.

Virtual Environments:

It's a good practice to use virtual environments to manage Python packages for different projects. You can create a virtual environment using the ``venv`` module (built into Python 3) or by using tools like ``virtualenv`` or ``conda``.

IDEs (Integrated Development Environments):

You may also want to use an IDE for Python development. Popular choices include PyCharm, VSCode, and Jupyter Notebooks.

4.5.3 IDLE

IDLE, the Integrated Development and Learning Environment, is a comprehensive software tool included with the standard distribution of Python, designed to facilitate the development and learning of Python programming. Serving as both an editor and an interactive shell, IDLE provides users with a versatile platform for writing, testing, and debugging Python code.

The heart of IDLE is its text editor, offering a user-friendly interface for creating and editing Python scripts. The editor incorporates syntax highlighting, a feature that color-codes different elements of the code, making it easier for users to identify and understand the structure of their programs. Additionally, IDLE's editor assists with indentation, a crucial aspect of Python syntax, ensuring that code blocks are visually clear and aligned correctly.

One of the standout features of IDLE is its interactive shell, which allows users to execute Python code in a line-by-line manner. This feature is particularly valuable for quick testing, experimentation, and interactive exploration of Python constructs. Users can input code directly into the shell, receive immediate feedback, and observe the results, making it an excellent environment for learning and prototyping.

IDLE also incorporates a powerful debugger, an essential tool for identifying and resolving issues within Python code. The debugger enables users to set breakpoints at specific lines of code, pause execution, and inspect variables to understand the program's state at different points. This capability is instrumental in locating and fixing bugs, enhancing the overall reliability and functionality of Python programs.

For learners and developers seeking guidance, IDLE integrates seamlessly with Python's extensive documentation through an in-built help system. This allows users to access information about modules, functions, and other Python constructs directly within the IDE. This feature enhances the learning experience and encourages users to explore the vast capabilities of the Python language.

In recent versions, IDLE has introduced additional features to enhance the overall development experience. A project browser has been integrated, aiding in the organization and navigation of code files within a project. This can be particularly useful for larger projects with multiple files, streamlining the development workflow.

While IDLE offers a convenient environment for beginners and those working on smaller projects, it may not provide the extensive features and tools required for larger and more complex development tasks. Advanced users often opt for dedicated Integrated Development Environments (IDEs) such as PyCharm, Visual Studio Code, or Jupyter Notebooks, which offer more sophisticated features like advanced code navigation, version control integration, and support for various programming languages.

IDLE's cross-platform compatibility ensures that users on Windows, macOS, and Linux can access a consistent development environment. Its simplicity and ease of use make it an excellent choice for educational purposes, introductory programming, and quick scripting tasks.

IDLE plays a crucial role in the Python ecosystem by providing an integrated and accessible environment for coding, testing, and learning. While it may not cater to the advanced needs of experienced developers, its user-friendly features make it an invaluable tool for beginners and those looking to quickly prototype and experiment with Python code.

4.5.4 Decision Tree Classifier

Decision Tree Classifier explained in real-life: picking a vacation destination Decision Tree is a Supervised Machine Learning Algorithm that uses a set of rules to make decisions, similarly to how humans make decisions. Decision Tree is a Supervised Machine Learning Algorithm that uses a set of rules to make decisions, similarly to how humans make decisions. One way to think of a Machine Learning classification algorithm is that it is built to make decisions.

You usually say the model predicts the class of the new, never-seen-before input but, behind the scenes, the algorithm has to decide which class to assign. Some classification algorithms are probabilistic, like Naive Bayes, but there's also a rule-based approach. We humans, also make rule-based decisions all the time.

When you're planning your next vacation, you use a rule-based approach. You might pick a different destination based on how long you're going to be on vacation, the budget available or if your extended family is coming along. The answer to these questions informs the final decision. And if you continually narrow down the available vacation destinations based on how you answer each question, you can visualize this decision process as a (decision) tree. A tree that makes decisions. Decision trees can perform both classification and regression tasks, so you'll see authors refer to them as CART algorithm: Classification and Regression Tree. This is an umbrella term, applicable to all tree-based algorithms, not just decision trees. But let's focus on decision trees for classification.

The intuition behind Decision Trees is that you use the dataset features to create yes/no questions and continually split the dataset until you isolate all data points belonging to each class. With this process you're organizing the data in a tree structure. Every time you ask a question you're adding a node to the tree. And the first node is called the root node.

Example of a decision tree with tree nodes, the root node and two leaf nodes. (Image by author) Every time you answer a question, you're also creating branches and segmenting the feature space into disjoint regions. One branch of the tree has all data points corresponding to answering Yes to the question the rule in the previous node implied. The other branch has a node with the remaining data points.

This way you narrow down the feature space with each split or branch in the tree, and each data point will only belong to one region. The goal is to continue to splitting the feature space, and applying rules, until you don't have any more rules to apply or no data points left. Then, it's time to assign a class to all data points in each leaf node. Assigning a class The algorithm tries to completely separate the dataset such that all leaf nodes, i.e., the nodes that don't split the data further, belong to a single class. These are called pure leaf nodes. But most times you end up with mixed leaf nodes, where not all data points have to the same class.

In the end, the algorithm can only assign one class to the data points in each leaf node. With pure leaf nodes that already taken care of, because all data points in that node have the same class. But with mixed leaf nodes the algorithm assigns the most common class among all data points in that node.

It's hard to build the perfect tree. The ideal tree is the smallest tree possible, i.e. with fewer splits, that can accurately classify all data points. This sounds simple, but it's actually a NP hard problem. Building the ideal tree would take polynomial time, which increases exponentially as the dataset grows.

For example, for a dataset with only 10 data points and an algorithm with quadratic complexity, $O(n^2)$, the algorithm executes $10 * 10 = 100$ iterations to build the tree. Expand that dataset a bit more to have 100 data points, and the number of iterations the algorithm will execute jumps to 10,000. Finding the best tree is ideal in theory, but as the dataset grows, it becomes computationally unfeasible! To turn this NP-hard problem into something computationally feasible the algorithm uses a greedy approach to build the next best tree.

Since it optimizes for local decisions, it focuses only on the node at hand, and what's best for that node in particular. So it doesn't need to explore all possible splits for that node and beyond. Picking the best split On every split, the algorithm tries to divide the dataset into the smallest subset possible. So, like any other Machine Learning algorithm, the goal is to minimize the loss function as much as possible.

A popular loss function for classification algorithms is Stochastic Gradient Descent but, it requires the loss function to be differentiable. So, it's not an option in this case. But since you're separating data points that belong to different classes, the loss function should evaluate a split based on the proportion of data points belonging to each class before and after the split. Decision Tree use loss functions that evaluate the split based on the purity of the resulting nodes. In other words, you'd want a loss function that evaluates the split based on the purity of the resulting nodes. A loss function that compares the class distribution before and after the split[2], like Gini Impurity and Entropy.

4.5.5 What Is Natural Language Processing:

Natural language processing (NLP) refers to the branch of computer science — and more specifically, the branch of artificial intelligence or AI — concerned with giving computers the ability to understand text and spoken words in much the same way human beings can.

NLP combines computational linguistics — rule-based modeling of human language — with statistical, machine learning, and deep learning models. Together, these technologies enable computers

to process human language in the form of text or voice data and to ‘understand’ its full meaning, complete with the speaker or writer’s intent and sentiment.

NLP drives computer programs that translate text from one language to another, respond to spoken commands, and summarize large volumes of text rapidly — even in realtime. There’s a good chance you’ve interacted with NLP in the form of voice-operated GPS systems, digital assistants, speech-to-text dictation software, customer service chatbots, and other consumer conveniences. But NLP also plays a growing role in enterprise solutions that help streamline business operations, increase employee productivity, and simplify mission-critical business processes.

Natural Language Processing (NLP) is a field of artificial intelligence that focuses on the interaction between computers and human language. It involves the development and application of algorithms and models that enable machines to understand, interpret, and generate human language. NLP plays a pivotal role in various applications, ranging from virtual assistants and chatbots to sentiment analysis, machine translation, and information extraction.

One of the fundamental challenges in NLP is the ambiguity and complexity inherent in human language. Natural languages are rich, context-dependent, and often include nuances, idioms, and cultural references. NLP systems aim to overcome these challenges by employing sophisticated techniques such as machine learning, deep learning, and statistical approaches to extract meaningful information from textual data.

In recent years, deep learning, particularly using recurrent neural networks (RNNs) and transformer models, has significantly advanced the capabilities of NLP. Models like BERT (Bidirectional Encoder Representations from Transformers) and GPT (Generative Pre-trained Transformer) have demonstrated state-of-the-art performance in various NLP tasks. These models leverage large pre-trained datasets to learn contextualized representations of language, enabling them to understand the context and semantics of words in a given sentence.

NLP finds applications in numerous industries and domains. In healthcare, NLP is used for clinical documentation, extracting information from medical records, and even predicting disease outbreaks based on textual data analysis. In finance, it aids in sentiment analysis of financial news and reports, contributing to investment decision-making.

Chatbots and virtual assistants are perhaps the most visible applications of NLP in everyday life. These systems leverage natural language understanding to comprehend user queries and generate relevant responses. They are used in customer support, information retrieval, and various interactive systems to provide users with a more natural and intuitive interaction experience.

However, despite the progress made, challenges in NLP persist, including dealing with ambiguous language, handling diverse linguistic expressions, and addressing bias in language models. Ongoing research and development in NLP aim to address these challenges, with a focus on creating more robust and ethically sound language processing systems.

In conclusion, Natural Language Processing is a dynamic and evolving field that continues to shape the way machines interact with and understand human language. Its applications are widespread, impacting industries ranging from healthcare and finance to customer service and beyond, with ongoing advancements driving the field towards more sophisticated and context-aware language processing capabilities.

4.5.6 NLP Libraries:

1. re (Regular Expressions):

``re`` is the module for regular expression operations. It provides support for regular expressions (regex), which are powerful tools for string manipulation and pattern matching.

The ``re`` library in Python provides support for regular expressions, enabling developers to work with patterns in strings. It offers functions like ``re.search()``, ``re.match()``, and ``re.findall()`` for searching and matching patterns within strings. The library also includes methods for pattern substitution using ``re.sub()``. The regex patterns can involve metacharacters, character classes, and quantifiers, allowing for flexible and powerful string matching.

Additionally, ``re`` supports grouping, making it easier to extract specific parts of matched patterns. Overall, the ``re`` library is a valuable tool for text processing and manipulation in Python, enhancing string-related tasks in various applications.

2. pandas:

``pandas`` is a powerful data manipulation and analysis library for Python. It provides data structures like DataFrame for efficient handling of structured data, and functions to read, write, and analyze data.

Pandas is a powerful open-source data manipulation and analysis library for Python. It introduces two primary data structures: DataFrame, a two-dimensional tabular structure, and Series, a one-dimensional labeled array. These structures facilitate easy manipulation and analysis of structured data, offering functions for cleaning, reshaping, and analyzing information.

Pandas excels in handling missing data, supports various file formats for data I/O, and provides extensive functionality for descriptive statistics, grouping, and aggregation. Widely embraced in data science and analytics, Pandas simplifies complex data tasks, making it an essential tool for Python developers and analysts alike.

3. pyttsx3:

`pyttsx3` is a Python library for text-to-speech conversion. It allows you to convert text into spoken words using various speech engines.

Pyttsx3 is a Python library that serves as a text-to-speech (TTS) conversion engine. It allows developers to incorporate speech synthesis capabilities into their Python applications seamlessly. The "3" in pyttsx3 denotes its compatibility with Python 3.x. This library supports multiple TTS engines, offering flexibility in choosing the one that best suits the developer's needs.

One of the key advantages of pyttsx3 is its simplicity and ease of use. Developers can convert text to speech with just a few lines of code, making it accessible for both beginners and experienced programmers. Pyttsx3 also provides control over speech rate, volume, and voice selection, allowing for customization to suit specific application requirements.

Moreover, pyttsx3 supports both synchronous and asynchronous speech synthesis, providing options for real-time interaction or background processing. It is platform-independent, making it compatible with various operating systems. With its straightforward integration and customization options, pyttsx3 empowers developers to enhance their applications by incorporating natural and dynamic speech synthesis capabilities.

4. sklearn (scikit-learn):

`scikit-learn` is a machine learning library for Python that provides simple and efficient tools for data analysis and modeling. It includes various algorithms for classification, regression, clustering, dimensionality reduction, and more.

Scikit-learn, commonly referred to as sklearn, is a comprehensive machine learning library for Python. It provides simple and efficient tools for data analysis and modeling, including various algorithms for classification, regression, clustering, and dimensionality reduction. Developed on NumPy, SciPy, and Matplotlib, sklearn offers a user-friendly interface for tasks such as model training, evaluation, and hyperparameter tuning.

The library supports standard machine learning workflows and includes utilities for data preprocessing, feature selection, and model validation. With extensive documentation and a large community, scikit-learn is a go-to choice for both beginners and experienced practitioners in the field of machine learning with Python.

5. numpy:

`numpy` is a fundamental package for scientific computing with Python. It provides support for large, multi-dimensional arrays and matrices, along with mathematical functions to operate on these arrays.

NumPy is a fundamental library for scientific computing in Python, providing support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays. Its key component is the NumPy array, a powerful data structure that facilitates efficient manipulation of numerical data. NumPy arrays are homogeneous and allow for vectorized operations, significantly enhancing performance compared to traditional Python lists.

The library is essential in various domains such as data science, machine learning, and numerical computing, offering tools for tasks like linear algebra, statistical analysis, and Fourier transforms. NumPy's seamless integration with other scientific libraries and its ability to interface with lower-level languages make it a cornerstone in the Python scientific computing ecosystem. Its open-source

NumPy, a pivotal library in Python, revolutionizes numerical computing by introducing the `numpy.ndarray` data type, facilitating efficient manipulation of large arrays and matrices. This library excels in mathematical operations, offering a comprehensive suite of functions that operate on entire arrays without the need for explicit loops, enhancing computational efficiency. NumPy's broadcasting capabilities further simplify operations on arrays with different shapes.

Additionally, the ``numpy.random`` module aids in generating random numbers and sampling from various probability distributions. NumPy's seamless integration with other scientific libraries like SciPy, pandas, and scikit-learn creates a powerful ecosystem, making it indispensable for professionals and researchers in physics, engineering, finance, machine learning, and data science, nature, extensive documentation, and active community contribute to its widespread adoption and continual development.

6. warnings:

The ``warnings`` module is part of the Python standard library and provides a way to handle warnings during code execution. It allows you to control the display of warning messages.

The ``warnings`` module in Python provides a way to handle warning messages generated during the execution of a program. Warnings are issued to alert developers about potential issues or deprecated features in their code. The ``warnings`` module allows users to control how these warnings are displayed or handled.

Key functions and features of the ``warnings`` module include ``warn()``, which manually triggers a warning, and ``filterwarnings()``, which allows users to modify the default behavior of warnings. Additionally, the ``simplefilter()`` function provides a simple way to set the filter action for warnings, and the ``showwarning()`` function allows customization of the warning message format.

Developers can use the ``warnings`` module to filter out specific warnings, ignore them, or convert them into exceptions, helping to maintain code quality and address potential issues proactively.

4.6 UML Diagrams

Unified Modeling Language (UML) is a standardized visual modeling language used in software engineering to document and design systems. UML provides a set of graphical notations for representing various aspects of a system, making it easier for stakeholders to understand, communicate, and analyze the system's structure and behavior. Here are some key aspects of UML diagrams:

Common Elements in UML Diagrams:

1. **Class:** Represents a template for creating objects and defines the properties (attributes) and behaviors (methods) of those objects.
2. **Object:** An instance of a class.
3. **Association:** Represents a relationship between classes, indicating that objects of one class are connected to objects of another class.
4. **Inheritance:** Indicates that one class inherits properties and behaviors from another class.
5. **Aggregation and Composition:** Represent different types of relationships between classes, showing how they are connected or composed.
6. **Multiplicity:** Specifies the number of instances of one class related to a single instance of the other class in an association.
7. **Dependency:** Represents a relationship where a change in one element may affect another element.

Benefits of UML Diagrams:

- **Communication:** UML provides a standardized way to communicate complex system designs among stakeholders.
- **Visualization:** Diagrams help in visualizing and understanding the structure and behavior of a system.
- **Analysis:** UML diagrams assist in analyzing and documenting requirements, design decisions, and system architecture.
- **Documentation:** Serve as valuable documentation for future reference and maintenance.

UML is a versatile and widely used tool in software development, helping teams to design, document, and understand complex systems. Different diagrams serve different purposes, and they can be used in combination to provide a comprehensive view of a software system.

4.6.1 USE OF UML DIAGRAMS

Complex applications need collaboration and planning from multiple teams and hence require a clear and concise way to communicate amongst them. Businessmen do not understand code. So UML becomes essential to communicate with non-programmers.

A lot of time is saved down the line when teams are able to visualize processes, user interactions and static structure of the system. UML is linked with object-oriented design and analysis. UML makes the use of elements and forms associations between them to form diagrams.

4.6.2. CLASSIFICATION OF UML DIAGRAMS

- **Structural Diagrams:** Capture static aspects or structure of a system. Structural Diagrams include: Component Diagrams, Object Diagrams, Class Diagrams and Deployment Diagrams.
- **Behavior Diagrams:** Capture dynamic aspects or behavior of the system. Behavior diagrams include: Use Case Diagrams, State Diagrams, Activity Diagrams and Interaction Diagrams.

Types of UML Diagrams:

1. **Class Diagram:** Represents the static structure of a system by showing classes, attributes, operations, and relationships.
2. **Use Case Diagram:** Describes the interactions between a system and its external actors, illustrating how the system responds to different stimuli.
3. **Sequence Diagram:** Shows the interactions between different objects or components over time, representing the flow of messages or actions.
4. **Activity Diagram:** Illustrates the workflow or activities in a system, highlighting the sequence of actions and decisions.
5. **State Diagram:** Represents the various states that an object or system can be in and the transitions between those states.
6. **Component Diagram:** Depicts the physical components of a system and their relationships, providing a high-level view of the system's architecture.
7. **Deployment Diagram:** Shows the physical deployment of software components on hardware nodes, indicating the distribution of processing across a network.
8. **Package Diagram:** Organizes and shows the dependencies between different packages in a system.
9. **Object Diagram:** Represents instances of classes and their relationships at a specific point in time, providing a snapshot of the system's state.

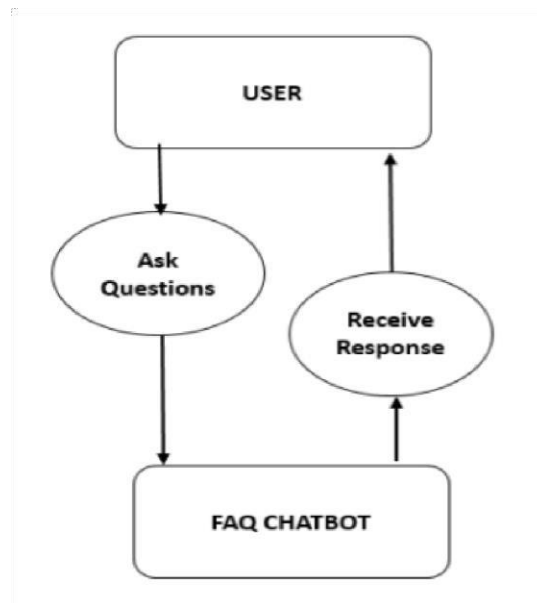


Figure 4.6.1 Class Diagram

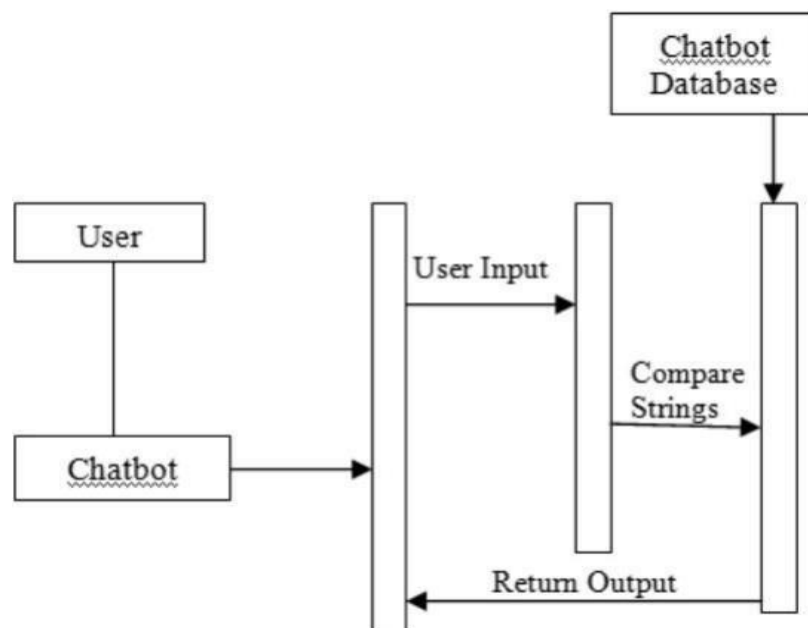


Figure 4.6.2 Sequence Diagram

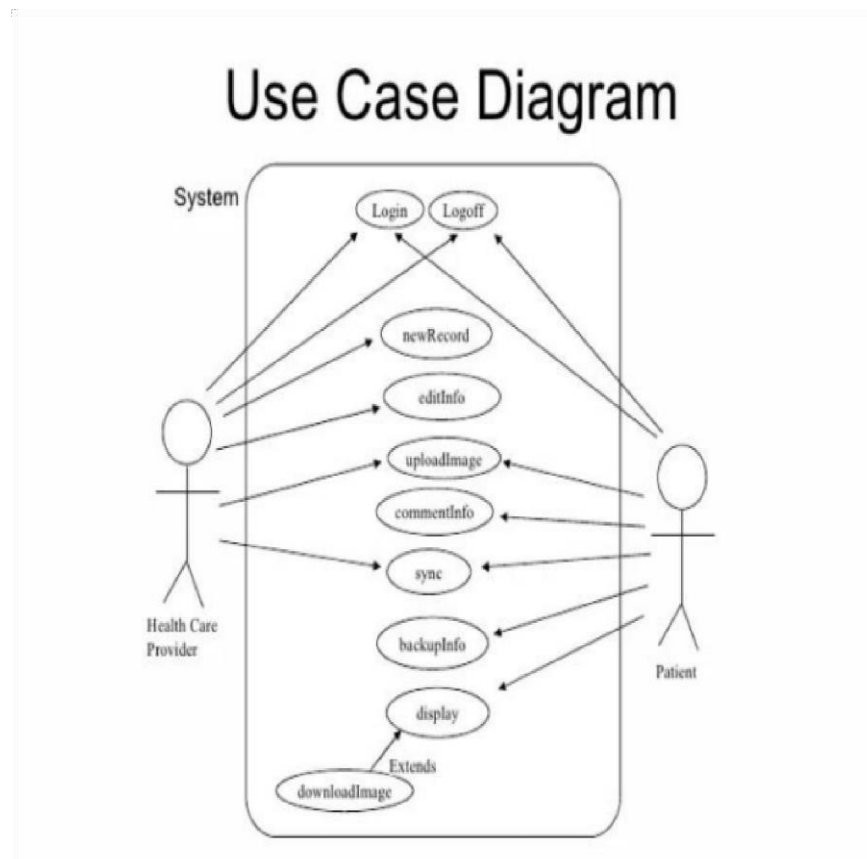


Figure 4.6.3 Use Case Diagram

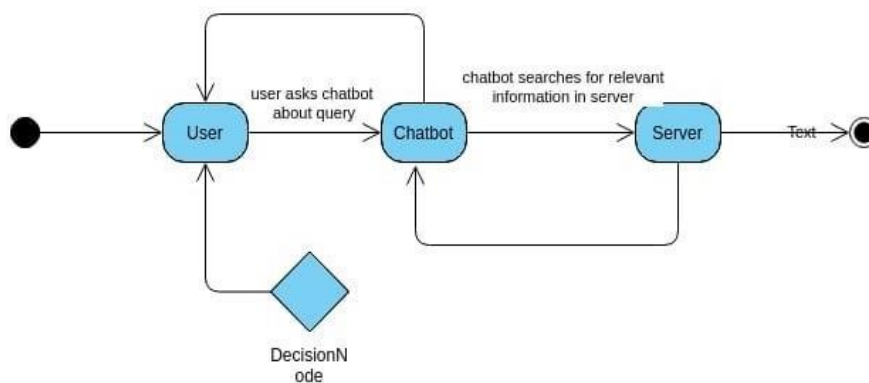


Figure 4.6.4 Activity Diagram

4.7 IMPLEMENTATION

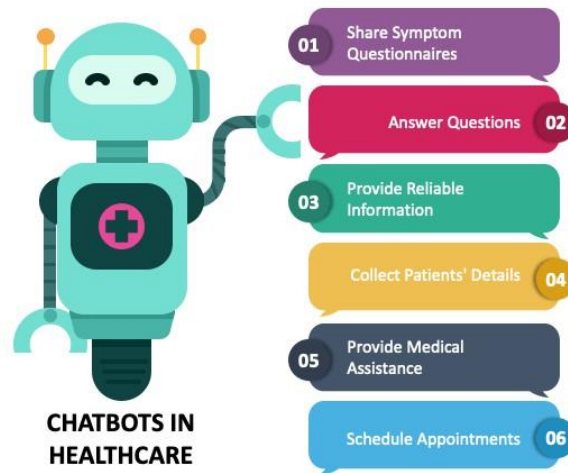
- 1. Importing Libraries:** The script begins by importing necessary libraries such as ``re``, ``pandas``, ``pyttsx3``, and various modules from `scikit-learn`.
- 2. Data Loading:** The script loads training and testing datasets using `pandas` from CSV files.
- 3. Data Preprocessing:** Label encoding is applied to the target variable ('prognosis') using `scikit-learn`'s ``LabelEncoder``. The data is split into training and testing sets.
- 4. Model Training:** Two machine learning models are trained - a Decision Tree Classifier (``clf1``) and a Support Vector Machine (``model``). Cross-validation scores are printed for the Decision Tree Classifier.
- 5. Feature Importance:** Feature importance are calculated for the Decision Tree Classifier, and the top features are displayed.
- 6. Text-to-Speech Output:** The script utilizes the ``pyttsx3`` library for text-to-speech output, making the interaction more user-friendly.
- 7. Symptom Information Retrieval:** The script reads additional information about symptoms, their descriptions, severities, and precautions from CSV files.
- 8. User Interaction:** The chatbot interacts with the user by asking for their name and inquiring about symptoms. It prompts the user to enter symptoms and the duration of symptoms.
- 9. Decision Tree Traversal:** The script traverses the Decision Tree based on the user's input symptoms, providing predictions, descriptions, and precautions.

Additionally, the code assumes that the Decision Tree model (``clf``) has already been trained and is available in the current session.

Before running this code, ensure that the required CSV files (``Training.csv``, ``Testing.csv``, ``symptom_Description.csv``, ``symptom_severity.csv``, ``symptom_precaution.csv``) are available in the specified paths, and the required libraries are installed. Additionally, the script uses a text-to-speech library (``pyttsx3``), which may need additional setup depending on your system.

CHATBOTS IN HEALTHCARE

Create Better Patient Experiences with AI-Powered Chatbots

**Figure 4.7.1: Uses of Chatbot****4.8 CODE**

```

import re
import pandas as pd
import pyttsx3
from sklearn import preprocessing
from sklearn.tree import DecisionTreeClassifier,_tree
import numpy as np
from sklearn.metrics import classification_report
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.svm import SVC
import csv
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

training = pd.read_csv('Data/Training.csv')

```

```
testing=pd.read_csv('Data/Testing.csv')
cols= training.columns
cols= cols[:-1]
x = training[cols]
y = training['prognosis']
y1= y

reduced_data = training.groupby(training['prognosis']).max()

le = preprocessing.LabelEncoder()
le.fit(y)
y = le.transform(y)

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.33, random_state=42)
testx  = testing[cols]
testy  = testing['prognosis']
testy  = le.transform(testy)

clf1 = DecisionTreeClassifier()
clf = clf1.fit(x_train,y_train)

scores = cross_val_score(clf, x_test, y_test, cv=3)
print (scores.mean())
model=SVC()
model.fit(x_train,y_train)
print("for svm: ")
print(model.score(x_test,y_test))

importances = clf.feature_importances_
indices = np.argsort(importances)[::-1]
```



```
features = cols
```

```
def readn(nstr):
```

```
    engine = pyttsx3.init()
```

```
    engine.setProperty('voice', "english+f5")
```

```
    engine.setProperty('rate', 130)
```

```
    engine.say(nstr)
```

```
    engine.runAndWait()
```

```
    engine.stop()
```

```
severityDictionary=dict()
```

```
description_list = dict()
```

```
precautionDictionary=dict()
```

```
symptoms_dict = { }
```

```
for index, symptom in enumerate(x):
```

```
    symptoms_dict[symptom] = index
```

```
def calc_condition(exp,days):
```

```
    sum=0
```

```
    for item in exp:
```

```
        sum=sum+severityDictionary[item]
```

```
    if((sum*days)/(len(exp)+1)>13):
```

```
        print("You should take the consultation from doctor. ")
```

```
        readn("You should take the consultation from doctor. ")
```

```
    else:
```

```
        print("It might not be that bad but you should take precautions.")
```

```
        readn("It might not be that bad but you should take precautions.")
```

```
def getDescription():
```

```
    global description_list
```

```
with open('MasterData/symptom_Description.csv') as csv_file:
    csv_reader = csv.reader(csv_file, delimiter=',')
    line_count = 0
    for row in csv_reader:
        _description={row[0]:row[1]}
        description_list.update(_description)
```

```
def getSeverityDict():
    global severityDictionary
    with open('MasterData/symptom_severity.csv') as csv_file:

        csv_reader = csv.reader(csv_file, delimiter=',')
        line_count = 0
        try:
            for row in csv_reader:
                _diction={row[0]:int(row[1])}
                severityDictionary.update(_diction)
        except:
            pass
```

```
def getprecautionDict():
    global precautionDictionary
    with open('MasterData/symptom_precaution.csv') as csv_file:

        csv_reader = csv.reader(csv_file, delimiter=',')
        line_count = 0
        for row in csv_reader:
            _prec={row[0]:[row[1],row[2],row[3],row[4]]}
            precautionDictionary.update(_prec)
```

```
def getInfo():
    print("-----HealthCare ChatBot-----")
    print("\nYour Name? \t\t\t",end="->")
    readn("\nYour Name? \t\t\t")
    name=input("")
    print("Hello,",name)
    readn("Hello,"+name)
```

```
def check_pattern(dis_list,inp):
    pred_list=[]
    inp=inp.replace(' ','_')
    patt = f"{inp}"
    regexp = re.compile(patt)
    pred_list=[item for item in dis_list if regexp.search(item)]
    if(len(pred_list)>0):
        return 1,pred_list
    else:
        return 0,[]
```

```
def sec_predict(symptoms_exp):
    df = pd.read_csv('Data/Training.csv')
    X = df.iloc[:, :-1]
    y = df['prognosis']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=20)
    rf_clf = DecisionTreeClassifier()
    rf_clf.fit(X_train, y_train)

    symptoms_dict = {symptom: index for index, symptom in enumerate(X)}
    input_vector = np.zeros(len(symptoms_dict))
    for item in symptoms_exp:
        input_vector[[symptoms_dict[item]]] = 1
```

```
return rf_clf.predict([input_vector])
```

```
def print_disease(node):
    node = node[0]
    val = node.nonzero()
    disease = le.inverse_transform(val[0])
    return list(map(lambda x:x.strip(),list(disease)))
```

```
def tree_to_code(tree, feature_names):
    tree_ = tree.tree_
    feature_name = [
        feature_names[i] if i != _tree.TREE_UNDEFINED else "undefined!"
        for i in tree_.feature
    ]
```

```
chk_dis=",".join(feature_names).split(",")
symptoms_present = []
```

```
while True:
    print("\nEnter the symptom you are experiencing \t\t",end="->")
    readn("Enter the symptom you are experiencing")
    disease_input = input("")
    cnf,cnf_dis=check_pattern(chk_dis,disease_input)
    if cnf==1:
        print("searches related to input: ")
        readn("searches related to input: ")
        num=0
        for num,it in enumerate(cnf_dis):
            print(num,")",it)
            readn(it)
        if num!=0:
```

```
print(f"Select the one you meant (0 - {num}): ", end="")
    readn(f"Select the one you meant (0 - {num}): ")
    conf_inp = int(input(""))
else:
    conf_inp=0

    disease_input=cnf_dis[conf_inp]
    break
else:
    print("Enter valid symptom.")
    readn("Enter valid symptom.")

while True:
    try:
        readn("Okay. From how many days ?")
        num_days=int(input("Okay. From how many days ? : "))
        break
    except:
        print("Enter valid input.")
        readn("Enter valid input.")
def recurse(node, depth):
    indent = " " * depth
    if tree_.feature[node] != _tree.TREE_UNDEFINED:
        name = feature_name[node]
        threshold = tree_.threshold[node]
        if name == disease_input:
            val = 1
        else:
            val = 0
        if val <= threshold:
            recurse(tree_.children_left[node], depth + 1)
```

```
else:
    symptoms_present.append(name)
    recurse(tree_.children_right[node], depth + 1)
else:
    present_disease = print_disease(tree_.value[node])

red_cols = reduced_data.columns
symptoms_given = red_cols[reduced_data.loc[present_disease].values[0].nonzero()]

print("Are you experiencing any ")
readn("Are you experiencing any ")
symptoms_exp=[]
for syms in list(symptoms_given):
    inp=""
    print(syms,"? : ",end="")
    readn(syms)
    while True:
        inp=input("")
        if(inp=="yes" or inp=="no"):
            break
        else:
            print("provide proper answers i.e. (yes/no) : ",end="")
            readn("provide proper answers i.e. (yes/no) : ")
    if(inp=="yes"):
        symptoms_exp.append(syms)

second_prediction=sec_predict(symptoms_exp)

calc_condition(symptoms_exp,num_days)
if(present_disease[0]==second_prediction[0]):
    print("You may have ", present_disease[0])
```

```
readn("You may have "+ present_disease[0])
print(description_list[present_disease[0]])
readn(description_list[present_disease[0]])
```

else:

```
print("You may have ", present_disease[0], "or ", second_prediction[0])
readn("You may have "+ present_disease[0]+ "or "+ second_prediction[0])
print(description_list[present_disease[0]])
readn(description_list[present_disease[0]])
print(description_list[second_prediction[0]])
readn(description_list[second_prediction[0]])
```

```
precaution_list=precautionDictionary[present_disease[0]]
print("Take following measures : ")
readn("Take following measures : ")
for i,j in enumerate(precaution_list):
    print(i+1,")",j)
    readn(str(i+1)+" "+j)
```

```
recurse(0, 1)
getSeverityDict()
getDescription()
getprecautionDict()
getInfo()
tree_to_code(clf,cols)
y_pred = clf.predict(x_test)
print(classification_report(y_test, y_pred))
svm_pred = model.predict(x_test)
print(classification_report(y_test, svm_pred))
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
svm_accuracy = accuracy_score(y_test, svm_pred)
```

```
print("SVM Accuracy:", svm_accuracy)
```

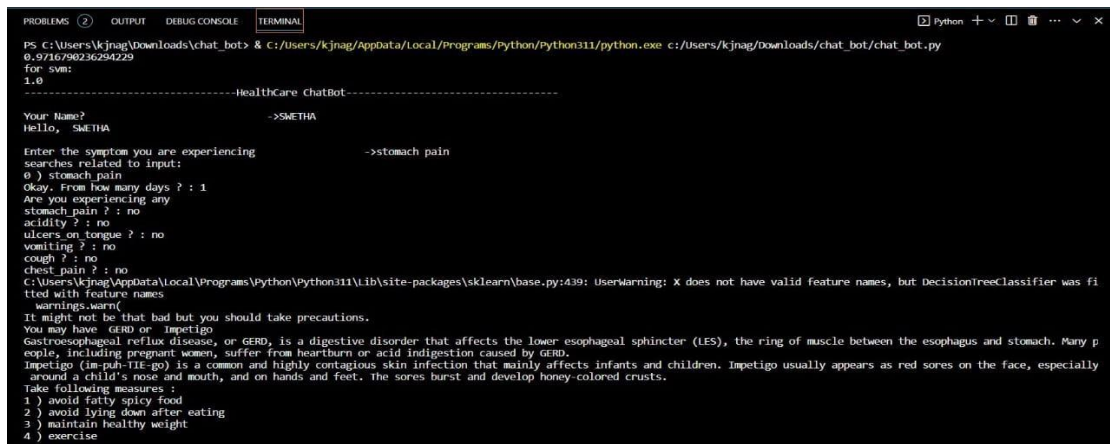
```
print("Thank You!")
```

```
readn("Thank You!")
```


CHAPTER 5

RESULT AND DISCUSSION

5.1 RESULT SCREENSHOTS



```

PS C:\Users\k\nag\Downloads\chat_bot> & C:/Users/k\nag/AppData/Local/Programs/Python/Python311/python.exe c:/Users/k\nag/downloads/chat_bot/chat_bot.py
0.9216/90236294229
for SVM:
1.0
-----HealthCare ChatBot-----
Your Name?                               ->SMETHA
Hello, SMETHA

Enter the symptom you are experiencing      ->stomach pain
searches related to input:
0 ) stomach_pain
Okay, From how many days ? : 1
Are you experiencing any
stomach_pain ? : no
acidity ? : no
ulcers on tongue ? : no
vomiting ? : no
cough ? : no
chest pain ? : no
C:\Users\k\nag\AppData\Local\Programs\Python\Python311\lib\site-packages\sklearn\base.py:439: UserWarning: X does not have valid feature names, but DecisionTreeClassifier was fi
tted with feature names
  warnings.warn(
It might not be that bad but you should take precautions.
You may have GERD or Impetigo
Gastroesophageal reflux disease, or GERD, is a digestive disorder that affects the lower esophageal sphincter (LES), the ring of muscle between the esophagus and stomach. Many p
eople, including pregnant women, suffer from heartburn or acid indigestion caused by GERD.
Impetigo (im-puh-TIE-go) is a common and highly contagious skin infection that mainly affects infants and children. Impetigo usually appears as red sores on the face, especially
around a child's nose and mouth, and on hands and feet. The sores burst and develop honey-colored crusts.
Take following measures :
1 ) avoid fatty spicy food
2 ) avoid lying down after eating
3 ) maintain healthy weight
4 ) exercise

```

Figure: 5.1 Stomach pain

The output generated by the chatbot, which follows a structured process involving user queries, information extraction, and a Decision Tree classifier algorithm, is aimed at providing users with valuable insights into disease prediction and precautionary measures. Let's delve into each step of the process to understand the output comprehensively.

Firstly, the user initiates interaction by entering a query into the chatbot window. This query could pertain to various health concerns, symptoms, or general inquiries about diseases. The chatbot, being designed to understand and process natural language, captures the user's input for further analysis.

Next, the chatbot employs a systematic approach to extract relevant details from the user's chat. This could involve identifying keywords, understanding the context, and extracting key information related to symptoms, medical history, or any other pertinent data. The goal is to gather a comprehensive understanding of the user's health-related concerns.

Subsequently, a Decision Tree classifier algorithm comes into play. This algorithm, known for its ability to make decisions based on a set of predefined rules, is employed to process the extracted information. In the context of disease prediction, the Decision Tree algorithm predictions about potential health issues.

```

-----HealthCare ChatBot-----
Your Name?                               ->Ram
Hello, Ram

Enter the symptom you are experiencing      ->fever
searches related to input:
0 ) high fever
1 ) mild fever
Select the one you meant (0 - 1): 0
Okay. From how many days ? : 2
Are you experiencing any
skin_rash ? : no
high_fever ? : yes
blister ? : no
red_sore_around_nose ? : no
yellow_crust_ooze ? : no
C:\Users\Kjnag\AppData\Local\Programs\Python\python311\Lib\site-packages\sklearn\base.py:439: UserWarning: X does not have valid feature names, but DecisionTreeClassifier was fi
tted with feature names
warnings.warn(
It might not be that bad but you should take precautions.
You may have Impetigo or (vertigo) Paroxysmal Positional Vertigo
Impetigo (im-puh-TIE-go) is a common and highly contagious skin infection that mainly affects infants and children. Impetigo usually appears as red sores on the face, especially
around a child's nose and mouth, and on hands and feet. The sores burst and develop honey-colored crusts.
Benign paroxysmal positional vertigo (BPPV) is one of the most common causes of vertigo at the sudden sensation that you're spinning or that the inside of your head is spinning
. Benign paroxysmal positional vertigo causes brief episodes of mild to intense dizziness.
Take following measures :
1 ) soak affected area in warm water
2 ) use antibiotics
3 ) remove scabs with wet compressed cloth
4 ) consult doctor
-----

```

Figure: 5.2 Fever

The output from the Decision Tree classifier is crucial in shaping the chatbot's response to the user. If the algorithm predicts a specific disease or health condition based on the input data, the chatbot can then provide the user with insights into the likelihood of that condition. This could include information about common symptoms associated with the predicted disease, potential causes, and perhaps even advice on seeking professional medical attention for a confirmed diagnosis.

Additionally, the chatbot may offer disease precaution recommendations. Based on the predicted disease or health condition, the chatbot can suggest precautionary measures that users can take to mitigate risks or manage symptoms. These precautions could include lifestyle changes, preventive actions, or guidance on when to consult a healthcare professional for further evaluation.

The overall output is a tailored response that combines disease prediction and precautionary advice. It is important to note that while the chatbot's predictions are generated through algorithmic analysis, they are not a substitute for professional medical advice. The chatbot's role is to provide users with information that may guide them in understanding potential health concerns and encourage them to seek professional medical assistance for a comprehensive evaluation.

The output of the chatbot is a user-specific response that leverages a Decision Tree classifier algorithm to predict diseases based on extracted information. The inclusion of precautionary measures enhances the practical utility of the chatbot, providing users with actionable advice to promote their health and well-being. However, users are always encouraged to consult healthcare professionals for personalized and accurate medical guidance.

5.2 PERFORMANCE EVALUATION

Evaluation appears to be an essential step in ensuring the rigor and relevance of a healthcare chatbot if it is to be adopted by practitioners in their day-to-day practice. As with medical devices, the answers provided by a chatbot should be validated scientifically and medically by a health authority. This would both build trust among users and certify the medical quality of such tools. In the following paragraphs, we would like to propose several key parameters that we believe are essential for effective evaluation of a chatbot.

Perceived relevance, contribution and usefulness

- Confidence in the answers, the completeness of the answers, the scope covered,
- Relevance to the topic addressed by the chatbot, accessibility to information, speed of access, clarity and tone used by the chatbot,
- Relevance: for decision-making, patient support,
- The contribution in practice and criteria, areas in which the tool can make a benefit,
- Impact: better assessment of the benefit-risk trade-off, awareness of the issues addressed.

Level of satisfaction/acceptability during demonstration/use

- Level of satisfaction rated on a scale from 0 to 10 (zero: not satisfied / 10: very satisfied),
- Dissatisfaction and satisfaction criteria,
- Reuse of the tool when available, reasons for reuse and non-reuse,
- Recommending the tool to other professionals.

We can evaluate the performance of our model using the below methods. There are different methods for both Decision Tree Classifier and support Vector Machine.

1. Decision Tree Classifier (DT):

After fitting the model, it uses cross-validation (``cross_val_score``) to assess the model's performance using ``scores.mean()`` for the mean of the cross-validation scores.

Metrics like accuracy, precision, recall, or F1 score can be computed and evaluated using ``cross_val_score`` or ``classification_report`` from scikit-learn, comparing predicted disease labels with the actual labels.

Evaluating Decision Tree Classifier:

After fitting the DT model (`clf`), use `cross\_val\_score` and define specific evaluation metrics.

For example:

```
from sklearn.metrics import classification_report
# Predict the test set
y_pred = clf.predict(x_test)
# Evaluate using classification report
print(classification_report(y_test, y_pred))
```

This code snippet will provide a detailed report including precision, recall, F1-score, and support for each class.

2. Support Vector Machine (SVM):

Similarly to evaluate accuracy of Support Vector Machine model('SVC') for disease prediction we can use 'model.score(x_test, y_test)'.

Evaluating Support Vector Machine (SVM):

Similar to the DT evaluation, you can utilize classification metrics to evaluate the SVM model as well.

```
from sklearn.metrics import classification_report
# Predict the test set
svm_pred = model.predict(x_test)
# Evaluate using classification report
print(classification_report(y_test, svm_pred))
```

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 CONCLUSION

The project represents a Healthcare ChatBot that employs machine learning models, specifically Decision Tree Classification and Support Vector Machines (SVM), to predict potential diseases based on user-input symptoms. The system utilizes a trained Decision Tree model to interpret symptoms and provide a probable disease diagnosis, while the SVM model offers an alternative approach for the same purpose.

This project integrates features for gathering symptom severity, associated descriptions, and precautionary measures for various ailments. The user interacts by inputting their symptoms, prompting the system to predict potential diseases and provide detailed descriptions along with recommended precautions. The code presents a comprehensive framework for a symptom-based disease prediction system that assists users in understanding possible health conditions based on their symptoms.

However, it could benefit from enhancements in the user experience, further validation, and a more extensive range of symptoms and diseases for a more robust and reliable prediction capability.

6.2 FUTURE ENHANCEMENTS

In the future, health care chatbots are poised for significant enhancements, primarily driven by advancements in technology and artificial intelligence. These improvements are expected to revolutionize the way users interact with these bots and receive health-related support. One crucial aspect of this evolution is the refinement of Natural Language Processing (NLP). Better contextual understanding, sentiment analysis, and an increased ability to grasp language nuances will enable chatbots to provide more accurate and meaningful responses to users' queries.

Another pivotal development on the horizon is the seamless integration of health care chatbots with Electronic Health Records (EHR). This integration will enable chatbots to access and update patients' medical information, ensuring a more personalized and informed interaction.

Users can expect chatbots to deliver tailored advice and recommendations based on their specific medical histories. As AI algorithms continue to progress, health care chatbots are likely to become more adept at preliminary diagnosis and treatment suggestions. The potential for these chatbots to analyze symptoms and offer informed recommendations could significantly impact the efficiency and accessibility of healthcare services.

Furthermore, the integration of advanced voice and speech recognition technologies will make these chatbots more user-friendly, allowing individuals to interact through voice commands, thereby increasing accessibility for those with mobility challenges or other limitations.

Real-time monitoring and alerts represent another avenue for improvement. By incorporating data from wearable devices and other sensors, health care chatbots could actively monitor users' health conditions. This would enable the provision of proactive health advice and timely alerts for potential issues, contributing to preventive healthcare.

To enhance the overall user experience, future health care chatbots may also evolve in their emotional intelligence and empathy. Incorporating sentiment analysis and responsive, empathetic interactions could make the interaction with chatbots more human-like and supportive, especially in situations where users are dealing with health-related concerns or emotional distress.

The future landscape of health care chatbots may also witness broader global accessibility through multi-language support. This development is crucial for catering to diverse linguistic backgrounds, making health care services more inclusive and user-friendly on a global scale.

As these advancements unfold, it will be essential to prioritize privacy and security measures to ensure the confidentiality of user data. The continuous learning and adaptation of chatbots, staying abreast of medical advancements, and adhering to evolving guidelines will be key to their effectiveness. Lastly, a seamless collaboration between health care chatbots and human professionals is paramount. These bots should facilitate smooth transitions to human doctors when necessary and maintain effective communication for a comprehensive and integrated approach to healthcare delivery. Together, these enhancements are poised to make health care chatbots powerful and intelligent allies in the provision of healthcare services.

CHAPTER 7

REFERENCES

1. Badlani, S., Aditya, T., Dave, M., & Chaudhari, S. (2021, May). Multilingual healthcare chatbot using machine learning. In *2021 2nd International Conference for Emerging Technology (INCET)* (pp. 1-6). IEEE.
2. Vineeth, R., Rithish, R., Sai Varma, D. V. S. N., & Ajay Prakash, B. V. (2020). Smart Health Care Chatbot for Prognosis of Treatments and Disease Diagnosis Using Machine Learning. *Journal of Computational and Theoretical Nanoscience*, 17(9-10), 3947-3951.
3. Kavitha, B. R., & Murthy, C. R. (2019). Chatbot for healthcare system using Artificial Intelligence. *Int J Adv Res Ideas Innov Technol*, 5, 1304-1307.
4. Shankhdhar, A., Adnekar, N., Bansal, P., & Agrawal, R. (2021). Healthcare Bot Using Machine Learning Algorithms for Medical Services. In *Inventive Systems and Control: Proceedings of ICISC 2021* (pp. 479-491). Springer Singapore.
5. PhaniRaghavaa, B., & Kumarb, S. A. (2022). An Improved Chatbot for Predicting Disease and Medicines Using Natural Language Processing with Fuzzy Logic. *Advances in Parallel Computing Algorithms, Tools and Paradigms*, 41, 258.
6. Tamizharasi, B., Livingston, L. J., & Rajkumar, S. (2020, December). Building a medical chatbot using support vector machine learning algorithm. In *Journal of Physics: Conference Series* (Vol. 1716, No. 1, p. 012059). IOP Publishing.
7. Bali, M., Mohanty, S., Chatterjee, S., Sarma, M., & Puravankara, R. (2019). Diabot: a predictive medical chatbot using ensemble learning. *International Journal of Recent Technology and Engineering*, 8(2), 6334-6340.
8. Bali, M., Mohanty, S., Chatterjee, S., Sarma, M., & Puravankara, R. (2019). Diabot: a predictive medical chatbot using ensemble learning. *International Journal of Recent Technology and Engineering*, 8(2), 6334-6340.